

품질공학설계

DOE + RSM + NSGA-II

이종수 교수님

이재욱 박사후연구원

jleej@yonsei.ac.kr, lee1jw@yonsei.ac.kr

연세대학교 기계공학부

2026. 09. 03.

1. DOE

실험계획법(CCD, D-optimal)

1. CCD

ccdesign(4, center=(0,0), alpha='r', face='ccc')

*(num of variables, center, alpha, face)

- face options → (ccc / cci / ccf)

```
[3] arr1 = ccdesign(4, center=(0,0), alpha='r', face='ccc')
```

```
[4] arr1
array([[ -1.,  -1.,  -1.,  -1.],
       [  1.,  -1.,  -1.,  -1.],
       [ -1.,   1.,  -1.,  -1.],
       [  1.,   1.,  -1.,  -1.],
       [ -1.,  -1.,   1.,  -1.],
       [  1.,  -1.,   1.,  -1.],
       [ -1.,   1.,   1.,  -1.],
       [  1.,   1.,   1.,  -1.],
       [ -1.,  -1.,  -1.,   1.],
       [  1.,  -1.,  -1.,   1.],
       [ -1.,   1.,  -1.,   1.],
       [  1.,   1.,  -1.,   1.],
       [ -1.,  -1.,   1.,   1.],
       [  1.,  -1.,   1.,   1.],
       [ -1.,   1.,   1.,   1.],
       [  1.,   1.,   1.,   1.],
       [ -2.,   0.,   0.,   0.],
       [  2.,   0.,   0.,   0.],
       [  0.,  -2.,   0.,   0.],
       [  0.,   2.,   0.,   0.],
       [  0.,   0.,  -2.,   0.],
       [  0.,   0.,   2.,   0.],
       [  0.,   0.,   0.,  -2.],
       [  0.,   0.,   0.,   2.]])
```

1. DOE

실험계획법(CCD, D-optimal)

2. D-optimal

```
def generate_design_space(num_factors, levels):
    design_space = np.array(list(product(levels, repeat=num_factors)))
    return design_space

def calculate_d_optimal_score(selected_samples):
    X = np.hstack((np.ones((selected_samples.shape[0], 1)), selected_samples))
    XT_X = np.dot(X.T, X)
    det_XT_X = np.linalg.det(XT_X)
    return -det_XT_X

design_space = generate_design_space(num_factors, levels)

temp = []

for i in range(100000):
    temp_space = design_space[np.random.choice(len(design_space)-1, num_samples, replace=False)]
    score = calculate_d_optimal_score(temp_space)

    temp.append([temp_space, score])

c = np.array(temp, dtype=object)

matrix = c[np.argmin(c[:,1]),0]
c_score = c[np.argmin(c[:,1]),1]
```

1. DOE

실험계획법 적용 예시(CCD)

예제

$$10 \leq x_1 \leq 15$$

$$15 \leq x_2 \leq 20$$

$$20 \leq x_3 \leq 30$$

$$25 \leq x_4 \leq 50$$

```
minimum = np.array([10, 15, 20, 25]); maximum = np.array([15, 20, 30, 50])
```

```
dfn = (df+1) / 2 * (maximum - minimum) + minimum
```

$$0 \leq y_{normalized} = \frac{(y - y_{minimum})}{(y_{maximum} - y_{minimum})} \leq 1$$

CCD array

CCD array

$$-1 \leq 2 \cdot y_{normalized} - 1 \leq 1$$

	x1	x2	x3	x4
0	-1.0	-1.0	-1.0	-1.0
1	1.0	-1.0	-1.0	-1.0
2	-1.0	1.0	-1.0	-1.0
3	1.0	1.0	-1.0	-1.0
4	-1.0	-1.0	1.0	-1.0



	x1	x2	x3	x4
0	10.0	15.0	20.0	25.0
1	15.0	15.0	20.0	25.0
2	10.0	20.0	20.0	25.0
3	15.0	20.0	20.0	25.0
4	10.0	15.0	30.0	25.0

Response Surface Methodology (RSM)

3. RSM

Math example : $y = 1x_1^2 + 2x_2^2 + 3x_3^2 + 4x_4^2$

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
degree = 2
```

```
poly_features = PolynomialFeatures(degree=degree, include_bias=True)
X_train_poly = poly_features.fit_transform(x_train) ←
```

```
lin_reg = LinearRegression()
lin_reg.fit(X_train_poly, y_train) ←
```

```
intercept : 4.440892098500626e-16 , coef: [-3.83720833e-15 -3.66547070e-15 6.22765728e-16 2.74780199e-15 1.00000000e+00 -
2.43728648e-15 4.75921386e-15 -5.51425225e-15 2.00000000e+00 -3.61689845e-16 5.93709110e-15 3.00000000e+00 3.26128013e-16
4.00000000e+00]
```

* parameters 순서: intercept, $x_1, x_2, x_3, x_4, x_1 * x_1, x_1 * x_2, x_1 * x_3, x_1 * x_4, x_2 * x_2, x_2 * x_3$...

```
def math_rsm(x1, x2, x3, x4):
    return np.sum(lin_reg.coef_[1:] * np.array([x1, x2, x3, x4, x1**2, x1*x2, x1*x3, x1*x4, x2**2, x2*x3, x2*x4, x3**2, x3*x4, x4**2]))
```

	x1	x2	x3	x4
0	10.0	15.0	20.0	25.0
1	15.0	15.0	20.0	25.0
2	10.0	20.0	20.0	25.0
3	15.0	20.0	20.0	25.0
4	10.0	15.0	30.0	25.0

➡
1D/3D
Simulation

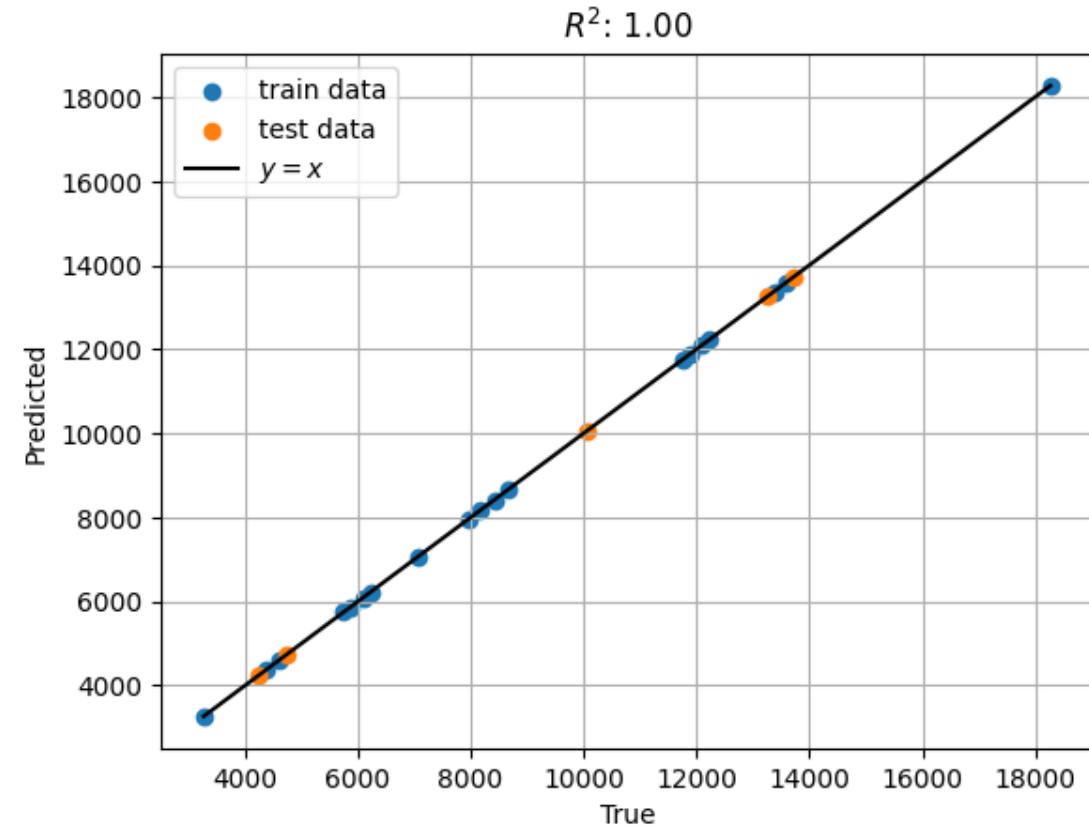
Y
데이터

Response Surface Methodology (RSM)

4. Evaluation

$$R^2 = \frac{SSR}{SST} = \frac{\sum (y_{pred} - y_{mean})^2}{\sum (y_{true} - y_{mean})^2}$$

```
plt.scatter(y_train, predict(x_train), label='train data')
plt.scatter(y_test, pred, label='test data')
plt.plot([min(output), max(output)], [min(output), max(output)], 'k',
label='$y=x$')
plt.legend()
plt.xlabel('True')
plt.ylabel('Predicted')
plt.title('$R^2$: {:.2f}'.format(r2))
plt.grid()
plt.show()
```



5. Import function (=RSM model)

```
def math_rsm(x1, x2, x3, x4):  
    return np.sum(lin_reg.coef_[1:] * np.array([x1, x2, x3, x4, x1**2, x1*x2, x1*x3, x1*x4, x2**2, x2*x3, x2*x4, x3**2, x3*x4, x4**2]))
```



```
def math_nsga(arr):  
    x1 = arr[:, 0]  
    x2 = arr[:, 1]  
    x3 = arr[:, 2]  
    x4 = arr[:, 3]  
    return np.array([math_rsm(x1[i], x2[i], x3[i], x4[i]) for i in range(len(x1))])
```

6. NSGA-II function

```
class func(Problem):
    def __init__(self):
        1. # of 설계변수, 목적함수, 제약조건
        super().__init__(n var=4, n obj=2, n ieq constr=1, vtype=float)
        self.xl = np.array([0, 0, 0, 0])
        self.xu = np.array([1.0, 1.0, 1.0, 1.0]) 2. Boundary → 설계 인자 범위

    def evaluate(self, x, out, *args, **kwargs):
        f1 = math_nsga(x) # objective
        f2 = 5 - math_nsga(x) 3. 목적함수
        4. 제약조건 ≥ 0
        g1 = math_nsga(x) - 2 # const, >=0
        # g2 = 9 * x[:, 0] - x[:, 1] - 1 # const, >=0

        out["F"] = anp.column_stack([f1, f2])

        out["G"] = anp.column_stack([g1]) 5. 함수 입력 (ex. 제약조건 2개: [g1, g2])
        out["G"] = - out["G"]
```


NSGA-II

7. Results

`res = minimize(problem, ... )`

1. 목적함수: `res.F`

2. 제약조건: `-res.G`

3. 설계인자: `res.X`

* Pareto optimal = ?

